

Lab 1:

	QUERY PLAN	
	text	
1	Nested Loop (cost=4.81..91.29 rows=32 width=21) (actual time=3.696..8.162 rows=32.00 loops=1)	
2	Buffers: shared hit=3 read=33	
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.477..0.488 rows=1.00 loops=1)	
4	Index Cond: (customer_id = 1)	
5	Index Searches: 1	
6	Buffers: shared read=3	
7	-> Bitmap Heap Scan on rental r (cost=4.53..82.68 rows=32 width=10) (actual time=3.202..7.623 rows=32.00 loops=1)	
8	Recheck Cond: (customer_id = 1)	
9	Heap Blocks: exact=28	
10	Buffers: shared hit=3 read=30	
11	-> Bitmap Index Scan on idx_fk_rental_customer_id (cost=0.00..4.53 rows=32 width=0) (actual time=0.283..0.284 rows=32.00 loops=1)	
12	Index Cond: (customer_id = 1)	
13	Index Searches: 1	
14	Buffers: shared hit=3 read=2	
15	Planning:	
16	Buffers: shared hit=199 read=16 dirtied=2	
17	Planning Time: 34.736 ms	
18	Execution Time: 9.555 ms	

1. **Vai trò của hai bảng:** Bảng customer là bảng ngoài (outer) vì được thực hiện trước bằng Index Scan. Bảng rental là bảng trong (inner), được truy cập bằng Bitmap Heap Scan kết hợp với index idx_fk_rental_customer_id.
2. **Lý do chọn Nested Loop:** Bảng customer chỉ trả về 1 dòng có customer_id = 1, sau đó PostgreSQL dùng index để tìm 32 bản ghi tương ứng trong bảng rental. Vì bảng ngoài chỉ có một dòng nên Nested Loop đơn giản và ít tốn chi phí hơn việc xây dựng hash table cho Hash Join.
3. **Ý nghĩa của loops=1:** Bảng trong được thực hiện một lần cho mỗi dòng của bảng ngoài. Do bảng customer chỉ trả về 1 dòng nên bảng rental cũng chỉ được truy cập 1 lần và trả về 32 dòng kết quả.

Lab 2:

	QUERY PLAN	
	text	
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=5.046..8.700 rows=1000.00 loops=1)	
2	Hash Cond: (f.language_id = lang.language_id)	
3	Buffers: shared read=89	
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=2.476..5.657 rows=1000.00 loops=1)	
5	Buffers: shared read=88	
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=2.528..2.530 rows=6.00 loops=1)	
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
8	Buffers: shared read=1	
9	-> Seq Scan on language lang (cost=0.00..1.06 rows=6 width=88) (actual time=2.501..2.506 rows=6.00 loo...	
10	Buffers: shared read=1	
11	Planning:	
12	Buffers: shared hit=281 read=10	
13	Planning Time: 49.812 ms	
14	Execution Time: 9.324 ms	

1. **Xác định hai giai đoạn:** Bảng language là bảng được dùng để xây dựng bảng băm vì nằm dưới node Hash và chỉ có 6 dòng. Bảng film là bảng được dò tìm, PostgreSQL quét 1.000 dòng của bảng này và đối chiếu language_id với bảng băm.
2. **Kiểm tra bộ nhớ:** Bảng băm sử dụng 9 kB bộ nhớ, gồm 1.024 bucket và chỉ có 1 batch. Điều này cho thấy toàn bộ bảng băm nằm vừa trong RAM và không phải ghi dữ liệu tạm xuống đĩa.

Hypothesis check:

	QUERY PLAN	
	text	
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.362..2.110 rows=1000.00 loops=1)	
2	Hash Cond: (f.language_id = lang.language_id)	
3	Buffers: shared hit=89	
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.317..1.298 rows=1000.00 loops=1)	
5	Buffers: shared hit=88	
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.029..0.030 rows=6.00 loops=1)	
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
8	Buffers: shared hit=1	
9	-> Seq Scan on language lang (cost=0.00..1.06 rows=6 width=88) (actual time=0.018..0.021 rows=6.00 loo...	
10	Buffers: shared hit=1	
11	Planning Time: 0.584 ms	
12	Execution Time: 2.256 ms	

sau khi set work_mem = 64kB 1

3. **Ảnh hưởng của work_mem = '64kB':** Sau khi giảm work_mem, PostgreSQL vẫn sử dụng Hash Join và bảng băm vẫn chỉ dùng 9 kB bộ nhớ với Batches: 1. Do 9 kB nhỏ hơn giới hạn 64 kB nên dữ liệu vẫn nằm hoàn toàn trong RAM và không phải ghi ra đĩa. Thời gian thực thi giảm từ 9,324 ms xuống 2,256 ms chủ yếu vì ở lần chạy thứ hai, 89 block dữ liệu đã có sẵn trong bộ nhớ đệm (shared hit=89), chứ không phải do giảm work_mem.

Lab 3:

	QUERY PLAN	
	text	
1	Merge Join (cost=412.64..622.36 rows=4581 width=23) (actual time=7.839..11.884 rows=4581.00 loops=1)	
2	Merge Cond: (f.film_id = i.film_id)	
3	Buffers: shared hit=86 read=26, temp read=10 written=12	
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.016..0.585 rows=1000.00 loo...	
5	Index Searches: 1	
6	Buffers: shared hit=83 read=1	
7	-> Materialize (cost=412.37..435.27 rows=4581 width=6) (actual time=7.808..9.646 rows=4581.00 loops=1)	
8	Storage: Memory Maximum Storage: 17kB	
9	Buffers: shared hit=3 read=25, temp read=10 written=12	
10	-> Sort (cost=412.37..423.82 rows=4581 width=6) (actual time=7.794..8.646 rows=4581.00 loops=1)	
11	Sort Key: i.film_id	
12	Sort Method: external merge Disk: 80kB	
13	Buffers: shared hit=3 read=25, temp read=10 written=12	
14	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=2.632..4.019 rows=4581.00 loops=1)	
15	Buffers: shared read=25	
16	Planning:	
17	Buffers: shared hit=86 read=5	
18	Planning Time: 30.260 ms	
19	Execution Time: 31.714 ms	

1. **Cấu trúc kế hoạch:** PostgreSQL đã sử dụng Merge Join. Bảng film được đọc bằng index film_pkey nên dữ liệu đã có thứ tự theo film_id và không cần sắp xếp lại. Trong khi đó, bảng inventory được quét tuần tự rồi sắp xếp theo i.film_id. Quá trình sắp xếp sử dụng phương pháp external merge và dùng 80 kB dung lượng đĩa tạm.

Reset lại

	QUERY PLAN	
	text	
1	Hash Join (cost=110.50..193.39 rows=4581 width=23) (actual time=0.568..2.316 rows=4581.00 loops=1)	
2	Hash Cond: (i.film_id = f.film_id)	
3	Buffers: shared hit=113	
4	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.027..0.385 rows=4581.00 loo...	
5	Buffers: shared hit=25	
6	-> Hash (cost=98.00..98.00 rows=1000 width=19) (actual time=0.530..0.530 rows=1000.00 loops=1)	
7	Buckets: 1024 Batches: 1 Memory Usage: 59kB	
8	Buffers: shared hit=88	
9	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=19) (actual time=0.015..0.243 rows=1000.00 loo...	
10	Buffers: shared hit=88	
11	Planning:	
12	Buffers: shared hit=6	
13	Planning Time: 0.594 ms	
14	Execution Time: 2.662 ms	

2. **So sánh:** Sau khi bật lại các thuật toán và bỏ order by, PostgreSQL chuyển từ Merge Join sang Hash Join. Hệ thống tạo bảng băm từ 1.000 dòng của bảng film, sử dụng 59 kB bộ nhớ, rồi quét 4.581 dòng của bảng inventory để đối chiếu. Hash Join được chọn vì không cần sắp xếp dữ liệu như Merge Join nên có chi phí thấp hơn. Thời gian thực thi cũng giảm từ 31,714 ms ở Lab 3AA xuống còn 2,662 ms.